

Entrega 4 - Hola mundo

Ángel David Suescun Romero
asuescunr@unal.edu.co

Laura Camila Perilla Torres
lperillat@unal.edu.co

Juan Pablo Rodríguez Briceño
jrodirguezbr@unal.edu.co

Wilson Franco Martinez
wfrancom@unal.edu.co

Asignatura: Ingeniería de software I

Docente: Oscar Eduardo Alvarez Rodriguez

Universidad Nacional de Colombia
Sede Bogotá

Bogotá D.C
Colombia

TABLA DE CONTENIDOS

Introducción.....	3
Instalación de Java.....	3
Instalación de Docker.....	3
Creación del proyecto.....	4
- Manejo de dependencias.....	5
Ejemplo de proyecto.....	6
- Usuario.java.....	6
- UsuarioRepository.java.....	6
- UsuarioService.java.....	7
- SaludoController.java.....	7
Cambio de archivo de recursos de Java.....	9
Creación de Base de Datos en MySQL.....	10
Creación del Dockerfile.....	10
Creación del contenedor con Docker.....	11

Introducción

En este tutorial se va a realizar una explicación de como iniciar un contenedor en Docker para montar un proyecto que use el lenguaje Java y el framework Spring Boot. Adicionalmente, se va a hacer uso de MySQL para la base de datos.

Instalación de Java

Antes de instalar java, se puede correr el siguiente comando en alguna terminal o en PowerShell:

```
java -version
```

Si el dispositivo a usar lo tiene instalado, saldrá algo similar a:

```
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode,
sharing)
```

De no ser así, se debe descargar la versión de Java de acuerdo al sistema operativo a través de este enlace:
<https://www.oracle.com/java/technologies/downloads/>

Instalación de Docker

Antes de instalar Docker, se puede correr el siguiente comando en alguna terminal o en PowerShell:

```
Docker --version
```

Si el dispositivo a usar lo tiene instalado, saldrá algo similar a:

```
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
```

De no ser así, se debe descargar el motor de Docker respectivo a la versión de linux a través de este enlace:

<https://www.oracle.com/java/technologies/downloads/>

Si se busca realizar la instalación para Windows o macOS, es necesario realizar primero la instalación de Docker Desktop. Se puede encontrar más información a través del siguiente enlace:

<https://docs.docker.com/desktop/>

Creación del proyecto

Teniendo en cuenta que queremos crear un nuevo proyecto con spring, podemos hacer uso de un generador de proyectos Spring Boot. Se puede acceder al enlace <https://start.spring.io/> para realizar esto.

Las opciones que se ven en la imagen son las recomendadas para el proyecto, con la excepción de la versión de Java, la cual debe coincidir con la versión de Java que se haya instalado.



The image shows the Spring Initializr web form for creating a new project. The form is dark-themed with green accents. It includes sections for Project, Language, Spring Boot, and Project Metadata. The selected options are: Project: Maven; Language: Java; Spring Boot: 4.0.6; Project Metadata: Group (com.example), Artifact (helloworld), Package name (com.example.helloworld), Packaging (Jar), Configuration (Properties), and Java (17).

Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1) ☐ 4.0.7 (SNAPSHOT) ☒ 4.0.6

☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☐ 21 ☒ 17

Una vez hayamos seleccionado las características del proyecto, procedemos a agregar las dependencias necesarias. Teniendo en cuenta que se quiere tener un proyecto API REST que use MySQL, se deben instalar las dependencias indicadas en la siguiente captura de pantalla:

Dependencies

ADD ...

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

MySQL Driver SQL
MySQL JDBC driver.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Entre las dependencias seleccionadas se hablará un poco de Spring Data JPA. Esta dependencia hace uso de la librería Hibernate, la cual es una librería ORM (Object-Relational Mapping) que permite una conexión entre la base de datos y la aplicación de una manera más amigable para el desarrollador, ya que evita que tengamos que manipular la base de datos a través de la aplicación usando cadenas que representen instrucciones SQL.

Una vez tenemos las propiedades y las dependencias del proyecto, podemos proceder a Generar el proyecto. Esto hará que la herramienta descargue un archivo con el nombre indicado en el campo de “Artifact” (De nombre “helloworld.zip” para este ejemplo).

Procedemos a descomprimir este archivo .zip en alguna carpeta de nuestro ordenador en el que tendremos el proyecto. Es importante remarcar que toda la información de las dependencias va a ser almacenada en el archivo pom.xml.

- Manejo de dependencias

Si se busca agregar, remover o modificar dependencias en un futuro, se debe modificar, agregar o eliminar el archivo a partir de la línea `<dependencies>`. Si se modifica este archivo, es importante ejecutar el siguiente comando para aplicar los cambios en las dependencias del proyecto:

```
mvn clean install
```

Ejemplo de proyecto

Por efectos de practicidad, se ponen varios archivos que deben ser incluidos en el proyecto:

- Usuario.java

```
package com.ejemplo.holamundo.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    @Column(nullable = false, unique = true)
    private String email;
}
```

Ruta del archivo: src/main/model/

- UsuarioRepository.java

```
package com.ejemplo.holamundo.repository;

import com.ejemplo.holamundo.model.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
}
```

Ruta del archivo: src/main/repository/

- UsuarioService.java

```
package com.ejemplo.holamundo.service;

import com.ejemplo.holamundo.model.Usuario;
import com.ejemplo.holamundo.repository.UsuarioRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class UsuarioService {
    @Autowired
    private UsuarioRepository usuarioRepository;

    public List<Usuario> obtenerTodos() {
        return usuarioRepository.findAll();
    }

    public Optional<Usuario> obtenerPorId(Long id) {
        return usuarioRepository.findById(id);
    }

    public Usuario crear(Usuario usuario) {
        return usuarioRepository.save(usuario);
    }

    public Usuario actualizar(Long id, Usuario usuarioActualizado) {
        return usuarioRepository.findById(id).map(usuario -> {
            usuario.setNombre(usuarioActualizado.getNombre());
            usuario.setEmail(usuarioActualizado.getEmail());
            return usuarioRepository.save(usuario);
        }).orElseThrow(() -> new RuntimeException("Usuario no encontrado"));
    }

    public void eliminar(Long id) {
        usuarioRepository.deleteById(id);
    }
}
```

Ruta del archivo: src/main/service/

- SaludoController.java

```
package com.ejemplo.holamundo.controller;
```

```
import com.ejemplo.holamundo.model.Usuario;
import com.ejemplo.holamundo.service.UsuarioService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/usuarios")
@CrossOrigin(origins = "*")
public class SaludoController {

    @Autowired
    private UsuarioService usuarioService;

    @GetMapping("/hola")
    public ResponseEntity<String> hola() {
        return ResponseEntity.ok("¡Hola Mundo desde Spring Boot!");
    }

    @GetMapping
    public ResponseEntity<List<Usuario>> obtenerTodos() {
        return ResponseEntity.ok(usuarioService.obtenerTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Usuario> obtenerPorId(@PathVariable Long id) {
        Optional<Usuario> usuario = usuarioService.obtenerPorId(id);
        return usuario.map(ResponseEntity::ok).orElseGet(() ->
ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Usuario> crear(@RequestBody Usuario usuario) {
        Usuario nuevoUsuario = usuarioService.crear(usuario);
        return ResponseEntity.status(HttpStatus.CREATED).body(nuevoUsuario);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Usuario> actualizar(@PathVariable Long id,
@RequestBody Usuario usuario) {
        Usuario usuarioActualizado = usuarioService.actualizar(id, usuario);
```



```

        return ResponseEntity.ok(usuarioActualizado);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        usuarioService.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}

```

Ruta del archivo: src/main/controller/

Cambio de archivo de recursos de Java

Es necesario modificar el archivo **application.properties**. En este archivo va a ir la información necesaria para que el driver de MySQL de java sepa a qué base de datos se debe conectar, al igual que el usuario y la contraseña del usuario a usar.

```

# application.properties

# Spring Boot Server
server.port=8080

# MySQL Configuration
spring.datasource.url=jdbc:mysql://mysql:3306/holamundo_db
spring.datasource.username=usuario_app
spring.datasource.password=contraseña123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA/Hibernate Configuration
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Logging
logging.level.root=INFO
logging.level.com.ejemplo.holamundo=DEBUG

```

Creación de Base de Datos en MySQL

Se puede trabajar una base de datos y generar el archivo .sql que permita replicar dicha base de datos. Este archivo debe llamarse init.sql y estar ubicado en la carpeta raíz del proyecto.

```
-- init.sql
-- Este archivo se ejecuta automáticamente al iniciar el contenedor
MySQL

USE holamundo_db;

-- Crear tabla de usuarios (Hibernate también la puede crear, pero
aquí explícitamente)
CREATE TABLE IF NOT EXISTS usuarios (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insertar datos de ejemplo (opcional)
INSERT INTO usuarios (nombre, email) VALUES
('Juan Pérez', 'juan@example.com'),
('María García', 'maria@example.com'),
('Carlos López', 'carlos@example.com');
```

Creación del Dockerfile

Dockerfile es un archivo sin extensión (es decir, sin .txt, .yaml o .xml) que debe ir en la carpeta raíz del proyecto. Su trabajo es indicar a Docker el cómo debe crear el contenedor y cómo debe correr SpringBoot.

```
# Dockerfile
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:resolve
COPY src ./src
RUN mvn clean package -DskipTests
```

```
# Stage 2: Runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Creación del contenedor con Docker

Docker requiere de un archivo de nombre **docker-compose.yml** (el cual debe estar en la carpeta raíz del proyecto) en donde se le indica las propiedades de la base de datos y del contenedor. Para ello, facilitamos el siguiente archivo:

```
# docker-compose.yml

version: '3.8'

services:
  mysql:
    image: mysql:latest
    container_name: helloworld_mysql

    environment:
      MYSQL_DATABASE: holamundo_db
      MYSQL_USER: usuario_app
      MYSQL_PASSWORD: contraseña123
      MYSQL_ROOT_PASSWORD: root123

    ports:
      - "8080:8080"

    volumes:
      - mysql_data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

    networks:
      - holamundo-network

  spring-app:
    build:
      context: .
```

```
dockerfile: Dockerfile
container_name: holamundo-app
depends_on:
  mysql:
    condition: service_healthy
ports:
  - "8080:8080"
environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/holamundo_db
  SPRING_DATASOURCE_USERNAME: usuario_app
  SPRING_DATASOURCE_PASSWORD: contraseña123
networks:
  - holamundo-network

volumes:
  mysql_data:

networks:
  holamundo-network:
    driver: bridge
```

Este archivo es necesario para que el contenedor se cree correctamente. Si se comete un error de cualquier tipo con este archivo, el contenedor se creará pero podrá generar un contenedor que no se podrá usar.

Una vez se tenga este archivo, se procede a abrir una terminal que esté ubicado en la carpeta raíz del proyecto y se ejecuta el siguiente comando:

```
docker-compose up --build
```

Este comando va a comenzar la creación del contenedor teniendo en cuenta los parámetros del archivo docker-compose.yml. Asimismo, va a crear la base de datos y correr el framework de SpringBoot.

Es posible que se requieran permisos de administrador para poder ejecutar este comando.